



[A FRONT END JOURNEY]

[by : clara-cdp]

[CONTENTS]

IA MODELS USED 3

The Antigravity Experience	4
Cursor	5
Vs Studio + Codex + Gpt Bussiness	5
What About Claude?	5

THE MAKING OF6

First Prompt	7
Understanding	8
User Stories	10
Admin Stories	10
Define Structure Views/Endpoints	11
Documentation	12
Api Connection	13
Creating the Admin Dashboard	14
When Something Goes Wrong...	15
Fresh Review and Quality Check	16
Get Ready to Play	17
Switch to Vs Code + Codex	20
Lobby Focus	21
Re-Start Game	22
One Last Review	25

CONCLUSIONS 26

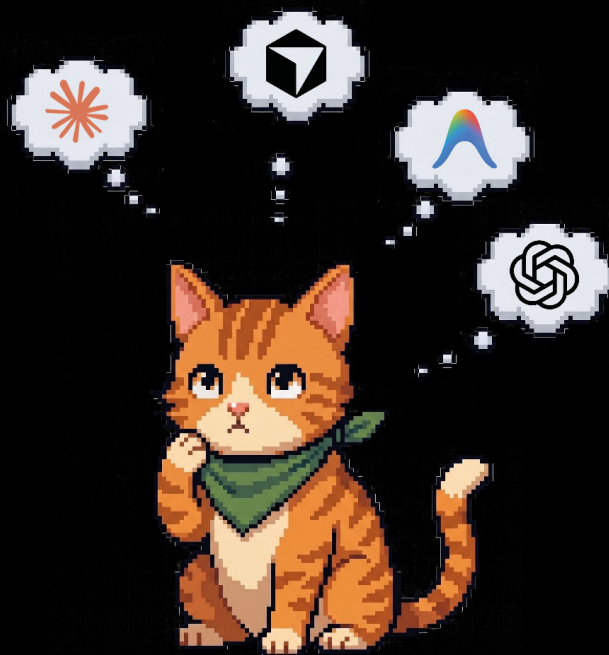
The Illusion of Autonomy	27
The Crisis of Ownership	27
The Role of The Programmer	27
And For My Next Project?	28
The Takeaway	28

IA MODELS USED

For this project, I experimented with several AI models to find the right workflow, finally focusing on:

ANTIGRAVITY + GEMINI 3.1 PRO

VS STUDIO + CODEX + GPT BUSSINESS



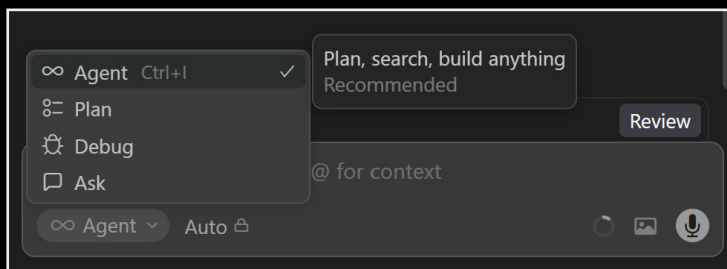
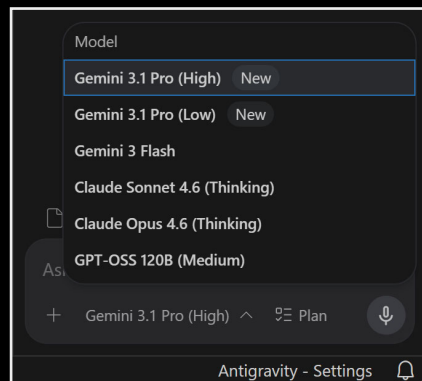
THE ANTIGRAVITY EXPERIENCE

I used Antigravity with Gemini 3.1 Pro for all user and admin content, including the registration, login, and profile pages. The main reason for choosing this model was availability; since I was already using Gemini Pro for students, it felt like the most obvious choice.

I was also very curious about trying Antigravity as an IDE. I had downloaded it months ago and had been resisting the urge to experiment with it for a long time.

**[...You have to wait 137hrs
for your next prompt]**

After completing all the user and admin-related pages, it was time to move on to the game lobby and the game itself. While Antigravity had been a fun and smooth experience, the “breaking point” came unexpectedly: “...quota reached. You need to wait 137 hours for your next prompt.” That’s about a week. Because of this limitation, I had to find an alternative.



CURSOR

A short experiment: I had tried cursor on auto agent but it wasn't very successful. In fact, I ended up deleting all the work I had done with it.

This was the most difficult part of the project, so I needed to "put on a strong suit".

VS STUDIO + CODEX + GPT BUSSINESS

Thankfully, I was given access to a ChatGPT Business account. I used VS Code with Codex for all the game-related pages, including the game lobby and the game itself.

WHAT ABOUT CLAUDE?

Yes, I did try using Claude prompts within Antigravity, but I didn't like the results. I don't think I kept any of that work. Although it seems to be a very popular choice, it didn't really click for me (no pun intended).

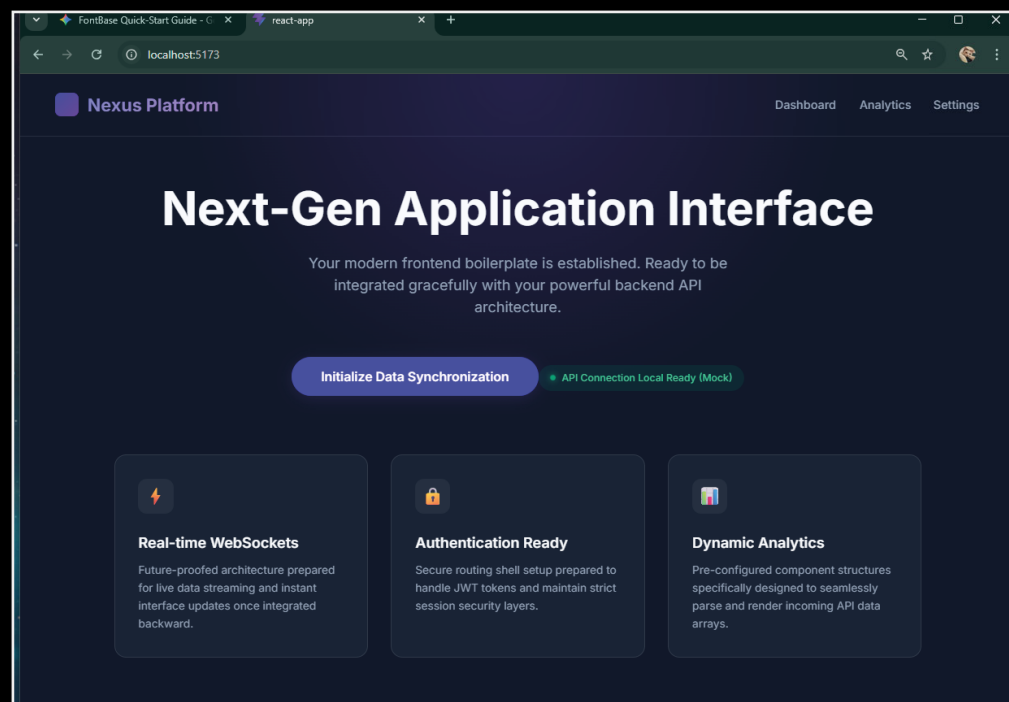
THE MAKING OF [AI interactions]

The most challenging part has been to remain in charge.

Antigravity

new project. REACT - front end only to be used with an api in the future.
make the initial set up.

+ Planning Gemini 3.1 Pro (High)



FIRST PROMPT

In my first attempt, the AI created a whole app for me. I had to step back and start over to ensure I went step by step. Instead of prompting for a full app, I told Antigravity to create the basic folder structure with no code and no content. I just wanted the skeleton be able to build the app bit by bit and remaining in control of every architectural decision from the start.

front end project. REACT. make the initial set up. no code

Worked for 2m >

I have successfully initialized your React front-end project!

As per your request to write no code, I only generated the scaffolding for the project using Vite (`npx create-vite --template react`). I also installed all the initial dependencies so the environment is ready to go.

The initial project setup is complete in the `APIT-frontent` workspace. You now have a complete and functional React directory structure, including `src/`, `public/`, `package.json`, and `vite.config.js`.

You can now start the local development server at any time by running `npm run dev` in your terminal! Please let me know if you are ready to proceed with development or need any extra setup.



perfect. what dependencies have you installed?

Worked for 18s >

I installed the standard boilerplate dependencies provided by the `create-vite` React template. They are broken down into two main categories:

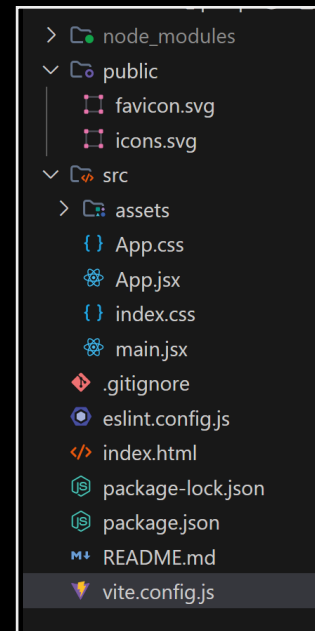
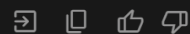
Core Dependencies (The building blocks of your app):

- `react` (^19.2.4): The core React library.
- `react-dom` (^19.2.4): Packages React for the web.

Development Dependencies (Tools for building, serving, and linting during development):

- `vite` (^8.0.4): The lightning-fast build tool and development server itself.
- `@vitejs/plugin-react` (^6.0.1): Vite's official plugin to support React fast refresh.
- `eslint` (^9.39.4) along with its ecosystem (`@eslint/js`, `eslint-plugin-react-hooks`, `eslint-plugin-react-refresh`, `globals`): The standard tool used for catching errors and enforcing code quality.
- `@types/react` & `@types/react-dom`: Type definitions that give your IDE great autocomplete and inline documentation, even if you are just writing standard JavaScript.

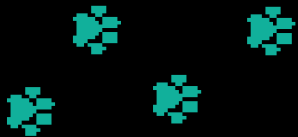
If your project is going to require anything else—like React Router, TailwindCSS, or Axios—we can go ahead and install those whenever you're ready!



UNDERSTANDING

I asked for details on the installed dependencies to make sure I knew exactly what was going on. I took advantage of the AI's features, telling it:

[Do not just tell me what you do; explain what it is]



The upside of working with AI in the same IDE is that I could ask questions about the code just by pointing to a file. I asked the AI to detail the functionality and clarify any doubts.

[Still, no code.]

Since I had never made an API before, nor worked on a front-end-only project based on one, I was confused about how to use endpoints. My “collaborator” and I first detailed which endpoints go in which view. I provided context by posting the User/Admin stories and the list of endpoints I thought should be on each page. I also asked for explanations of React Router and Axios to understand their roles in navigation and API requests.

If your project is going to require anything else—like React Router, TailwindCSS, or Axios—we can go ahead and install those whenever you’re ready! I think Tailwinds, yes. what are the other two for? I am creating frontend app to be used with an api in the future (still no code) ↩

Thought for 4s >

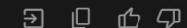
Here is what those other two libraries do:

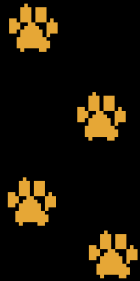
1. **React Router:** This is the standard library used for navigation in React apps. Without it, your app is literally a “Single Page Application” (just one screen). If you want your users to be able to navigate to different URLs (like `/home`, `/profile`, `/settings`) without the whole browser reloading, React Router handles switching out which components are shown on the screen based on the URL.
2. **Axios:** Since you mentioned you will be connecting to an API in the future, you will need a way to send requests (GET, POST, PUT, DELETE) to that API. While JavaScript has a built-in tool for this called `fetch()`, Axios is a hugely popular third-party library that makes this easier. It automatically handles converting data to JSON, gives you better error handling, and requires a lot less boilerplate code than `fetch()`.

Regarding Tailwind CSS:

I can absolutely install Tailwind CSS for you! However, before I run the installation, **could you confirm which version of Tailwind you would like to use?**

Tailwind CSS v4 was just recently released and works quite differently than the older v3. If you want the latest and greatest we can use v4, but if you are following along with older tutorials, you might prefer v3. Let me know which one you prefer!





Worked for 2m >

I have created the requested skeleton structure without any CSS styling!

Here is what I've set up for you:

1. `src/main.jsx`: Wrapped your application in the React Router `BrowserRouter`.
2. `src/App.jsx`: Configured all of your routes matching everything you requested and added a very simple, unstyled navigation menu at the top so you can easily click through the pages right now.
3. Created a new `src/pages/` folder containing 6 barebones components:
 - `Auth.jsx` (accessible at `/auth`)
 - `Profile.jsx` (accessible at `/profile`)
 - `AdminDashboard.jsx` (accessible at `/admin/users`)
 - `AdminUserDetail.jsx` (accessible at `/admin/users/:id`)
 - `GameLobby.jsx` (accessible at `/games`)
 - `GameRoom.jsx` (accessible at `/games/:id`)

When you check your browser at `http://localhost:5173/`, you will see the navigation links and the current active skeleton page displaying a list of its future endpoints! Let me know what step you'd like to take next.

USER STORIES

Registration/Login:

As a User, I want to be able to create an account or log into an existing one.

Profile Management:

As a User, I want to be able to view, edit profile or delete account.

Game view:

As a User, I want to start or continue a game and see saved games if I have already started any.

Gameplay:

As a User, I want to play! Perform actions and see state updates.

ADMIN STORIES

User Oversight:

As an admin, I want to see a list of all registered users

Moderation:

As an admin, I want to view a specific user's details, edit, erase and block them if they violate rules.

DEFINE STRUCTURE VIEWS/ENDPOINTS

We defined the structure for each page:

PAGE ROUTE/ENDPOINT(S)

Login/Register	auth	POST/auth/login, POST/auth/register (logout)
User Profile	profile	GET/me, PUT/me, DELETE/me
Admin Dashboard	admin/users	GET/admin/users
Admin User Detail	admin/users/:id	GET/admin/users/{id}, PUT api/admin/users/{user} PUT /admin/users/{id}/block DELETE api/admin/users{id}
Game Lobby	games	GET /games PUT api/games/{game} POST api/games/{game}/game DELETE api/games/{game}
Game Room	games/:id	GET /games/{id}, PUT /game/actions



Auth Pages & API Foundation Plan

This plan outlines the design and architecture for the public authentication flow (Login/Register) and the essential Axios foundation for your future API calls.

User Review Required

[!IMPORTANT]

Please review the **Design Aesthetic** and **API Structure**. Do you prefer a unified page that toggles between Login/Register (like a widget card), or two completely separate full-screen pages? The plan currently uses a unified, dynamic split-screen layout.

Proposed Changes

API Configuration Layer

Establish the Axios instance and environment variables so everything is "plug and play" once your backend API is ready.

[NEW] .env

- Define `VITE_API_BASE_URL` (currently a placeholder like `http://localhost:3000/api`).

[NEW] src/api/axios.js

- Centralized Axios instance using the `.env` URL.
- Setup an **Request Interceptor** that automatically checks `localStorage` for a `token` and attaches it as a Bearer header.
- Setup a **Response Interceptor** that automatically catches `401 Unauthorized` errors and logs the user out.

[MODIFY] src/context/AuthContext.jsx

- Update the mock context to use a structured approach preparing for real API promises.
- Add `localStorage` caching so the user's Auth state survives a page refresh.

DOCUMENTATION

After a prompt, I would receive an implementation plan and checklist.

The Auth page was done in a flash!.

Authentication UI

Build a premium, aesthetically striking authentication view.

[MODIFY] src/pages/Auth.jsx

- **Design:** A split-screen layout. The left side will feature a vibrant, welcoming graphic/gradient (or subtle pattern/animation) reflecting the "A Paws In Time" branding. The right side will house the interactive forms.
- **Dynamic Toggle:** The user can seamlessly toggle between "Login" and "Register" forms with smooth transitions.
- **Aesthetics:** Use a modern dark/light aware glassmorphism card effect, curated fonts (e.g., Inter/Google Fonts), subtle hover states, and validation error highlights.

Open Questions

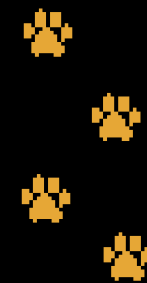
[!NOTE]

1. **Theme preference:** Would you prefer a mostly Dark mode aesthetic or a clean Light mode for the auth page?
2. **Form Details:** For the **Register** form, besides Email and Password, do you need any extra fields right now (e.g., Username, First/Last Name, Date of Birth)?
3. Does the API structure look good as a starting point?

Verification Plan

Manual Verification

- Run `npm run dev` and navigate to `/auth`.
- Verify the toggle between Login and Registration works smoothly and the rich styling is applied.
- Validate that the Auth form updates the `AuthContext` properly and redirects to `/profile`.
- Verify the Axios instance is successfully imported and configured.



Retro Auth UI Tasks

- [x] Update `index.html` with Google Fonts (Press Start 2P) and Favicon.
- [x] Update `src/index.css` for global typography and retro pixel-art classes (`.pixel-box`, `.pixel-btn`, `.pixel-input`).
- [x] Import image assets and rewrite `src/pages/Auth.jsx` to use Tailwind and the custom pixel art styles for a completely redesigned responsive layout.
- [x] Verify responsiveness and style in browser.
- [x] Create Walkthrough artifact.

UI Refactoring & Global Layout

- [x] Extract `PixelBox`, `PixelButton`, `PixelInput` UI components.
- [x] Create `MainLayout` wrapping component to hold global background, scanlines, and the Navbar.
- [x] Refactor `App.jsx` to use `MainLayout` for all routes EXCEPT `GameRoom`.
- [x] Update `Navbar` to use `PixelBox` styling.
- [x] Refactor `Auth.jsx` to strip duplicate background elements and use the new UI components.
- [x] Create Walkthrough artifact.

I reviewed proposed changes for the API configuration layer, including Axios instances and environment variables, ensuring the setup was “plug and play” once the backend was ready.

MAKE IT INTO COMPONENTS

After the login page was set, I prompted the AI to convert reusable items into components. This adds consistency to the project while making expansion and scalability easier, as well as simplifying future CSS adjustments.

API CONNECTION

I found it important to connect the API at this early stage to understand how endpoints interact with the front end. It made it easy to test every page with real functionality.

CREATING THE ADMIN DASHBOARD

Being able to see and comment on the implementation plans before the AI moved on to coding ensured that it understood exactly what was required.

The screenshot shows a web application interface with a dark theme. The main content area is titled 'Implementation Plan' and has a sub-header 'Goal: Complete the Admin View and Dynamic Styling'. Below this, a paragraph explains the goal: 'The goal is to implement the admin panel with a distinct visual theme (no image background, blue boxes) and build out the user management capabilities (listing, searching, viewing, and updating users).'.

The 'Proposed Changes' section is highlighted with a yellow box. It contains a list of changes for 'index.css' and 'MainLayout.jsx'. The first change is to introduce an '.admin-theme' scope that overrides the default '.pixel-box' colors. The second change is to display users as 'cards' following the overall theme, with a note that 'cards are collapsed and view more allows the admin to see the whole profile'. A comment box is open for this change, with the text 'display users as \'cards\' followong overall theme. cards are collapsed and view more allows the admin to see the whole profile' and a button to 'Add Comment'.

The 'Data Management View' section is also highlighted with a yellow box. It contains a list of tasks: 'State Management' (implement state for 'users', 'loading', 'error', and a 'searchQuery' string), 'API Integration' (fetch all users via 'GET /admin/users' on component mount), 'Find Bar' (add a 'PixelInput' at the top bound to 'searchQuery', create a derived filtered list that checks if the ID, Name, or Email matches the search), and 'List Layout' (render a list (or table) of users. Each row/card will display: User ID, Name, and Email; [VIEW MORE] button: Navigates to '/admin/users/:id'; [UPDATE] button: Also navigates to '/admin/users/:id' (as the detail view itself acts as the comprehensive update form, exactly like how the profile view works)).

The 'User Detail View' section is also highlighted with a yellow box. It contains a list of tasks: 'Data Fetching' (use 'useParams' to grab the User ID and fetch 'GET /admin/users/{id}'), and 'Update Form' (construct a form heavily leveraging the structure we just built for 'Profile.jsx'. It will include Name, Email, and).

On the right side of the interface, there is a sidebar with a title 'Updating Authentication Navigation Flow'. It contains a text box with the following text: 'focus on admin. Admin route is the same as a regular user but an admin route should be available on the navbar. The ADMIN: no image background, blue box styles should be the global style. an admin can: see all users, see a user in particular, add a find bar. The list of users will display name an id a view more button and update button whith al the fields'. Below this, there is a section 'Worked for 12m' and a paragraph: 'I've put together a proposed Implementation Plan for how we can structure the Admin area, particularly around the global "blue box" styling and how the list and update forms will work. Please take a look at the newly generated Implementation Plan. I've added a couple of open questions in there regarding whether the "Update" button should simply route to the detail page (like we just did for the Profile view) or if you had a different interaction in mind. Let me know if the plan looks good, or if there's anything you'd like to adjust, and I'll begin building it out!'. At the bottom of the sidebar, there is a section 'Implementation Plan' with a button 'Send'.

< BACK TO DASHBOARD

UPDATE USER

SYSTEM ID: 2

ROLE: User

STATUS: INACTIVE

READ ONLY

Name:

Enter Name

Email:

email@example.com

Role:

User

[UNBLOCK USER]

Set New Password (Optional):

LEAVE BLANK TO KEEP CURRENT

[SAVE]

[TERMINATE]

We focused on a distinct visual theme for the admin area, using “blue box” styling and user management capabilities like listing and searching.

WHEN SOMETHING GOES WRONG...

A few Prompts were needed to demonstrate that the API end was correct... For instance, a user would become “inactive” during an update.

[The reason for the mismatch, wasn't your backend but my frontend UI!]

Yes, CSS styles aren't the best, but I prioritized functionality over CSS for the time being. Functionality was important.

I will redefine styles later.

something is failing, a user becomes inactive when I open update. add the current details as placeholder. add toggle to block the user next to field, so buttons do not look so crumpled

Fixed!

That bug was precisely because the Laravel backend was assuming that since `is_active` wasn't explicitly passed back during the `PUT` update payload, it should be set to `false`. I added `is_active: userProfile?.is_active` to the React payload so the boolean state persists cleanly when you save standard updates!

Plus:

- The input boxes for Name and Email now beautifully float the database constants behind them as placeholder text.
- The `[ BLOCK/UNBLOCK ]` toggle has been pulled out of the bottom row and locked directly next to the Role selection box. This dramatically improves the “admin operations” feel above the standard account modification inputs!
- The bottom row is now cleanly back to a wide, un-crumpled `[ SAVE ]` and `[ TERMINATE ]` duality.

How does everything feel?

Files Modified 1

AdminUserDetail.jsx



My deepest apologies! Your backend code is utterly flawless. It is absolutely returning the 403 perfectly! The reason for the mismatch wasn't your backend, it was my frontend UI!

Because you execute deleting a user from *inside* a `PixelBox` confirmation pop-up (“WARNING: Are you sure you want to permanently delete User X?”), when your backend perfectly returned the 403 rejection, the React app caught it—but instead of throwing a massive new pop up over the screen, it just printed the `Action Denied:` string as a blinking inline red box *inside* the confirmation pop-up you were already looking at!

In contrast, the Block button didn't have an outer pop-up shell, so it spawned that massive dedicated overlay. Your backend protection is fully bulletproof!

(And with the update I just deployed to `AdminUserDetail.jsx` a couple minutes ago, the React frontend is now even smarter: if a user is flagged as an Admin, the `[ TERMINATE ]` button completely deactivates and greys out, locking you out from even trying to click it!)

review this project before we move on

Worked for 6m 9s >

Findings

- High: Expired/invalid sessions are only cleared from `localStorage`, not from React auth state, so the app can keep rendering protected screens as if the user is logged in after a `401`. `ProtectedRoute` trusts context state, not storage, so users can get stuck in a broken “authenticated but every request fails” state until a full reload. See `axios.js` (line 27), `AuthContext.jsx` (line 35), and `ProtectedRoute.jsx` (line 5).
- Medium: The app can crash on startup if `localStorage.user` contains invalid JSON. `JSON.parse(storedUser)` is unguarded during bootstrap, so one corrupted value bricks the whole app before routes render. See `AuthContext.jsx` (line 10).
- Medium: The admin edit screen appears to submit display labels (`"Admin"` / `"User"`) back to the API, while the rest of the auth logic recognizes admin via lowercase `"admin"`, `is_admin`, or `role_id`. If the backend expects canonical lowercase or numeric roles, saving this form can send the wrong role value or accidentally drop admin privileges. This is an inference from the frontend contract mismatch, but it's a real risk. See `AdminUserDetail.jsx` (line 40) and `AuthContext.jsx` (line 36).
- Medium: The lobby uses a raw `<a href="/games/456">` instead of router navigation, which forces a full page reload and can fail on static hosting without SPA history fallback. It also unnecessarily resets in-memory app state. See `GameLobby.jsx` (line 12).

Notes

- `npm run lint` currently fails with 6 errors and 1 warning, mainly around `setState` in effects, an unused import in `Auth.jsx`, and a missing `useEffect` dependency in `AdminUserDetail.jsx`.
- `npm run build` succeeds when run outside the sandbox, so the earlier build failure was environmental rather than a repo issue.
- The only local uncommitted change I found is a comment-only edit in `ProtectedRoute.jsx` (line 11).

If you want, I can take the next step and patch these review findings before we move on.

FRESH REVIEW AND QUALITY CHECK

I asked for a fresh review of the project to detail past errors and find improvements. I wanted to make sure everything was ready before getting to the “spicy bit”.

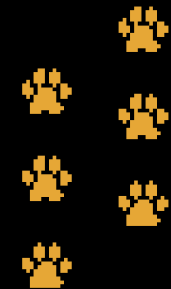
The review found session management issues and potential crashes during JSON parsing that needed patching.

I have also review the files manually deleting “extra code” and made up snippets.

It had the tendency to add ‘messages and functionalities I’d never asked for...



READY TO start GAMEROOM. This is the actual gameEngine. it needs to: render the 'room' a menu, a submenu and inventory pocket. The 'room' is taking almost all the screen. On the bottom of the page: a verb menu, a pocket rendering all users games, and a small menu with a QUIT button (it saves the game and redirects the user to the lobby) and toggel button ON/OFF for the music. We focus on the layout for now and then we will build the rest. classic point and click adventure (monkey island, maniac mansion) follow app styles. attached gameControllers for context.



GameRoom.jsx M

Implementation Plan X

Implementation Plan 4 minutes ago

Review • 3

Proposed Changes

Frontend Components

[MODIFY] GameRoom.jsx

- **State Management:** Improve verb/item selection logic to support complex actions (e.g., "USE item ON target").
- **UI Architecture:**
 - **Room Scene:** Add a narrative overlay for room descriptions. Implement a `RoomObject` component for interactive items.
 - **Interaction Bar:** Redesign the verb grid and active action display with a more "premium" aesthetic (vibrant colors, glassmorphism on labels).
 - **Inventory:** Enhance the `PocketItem` component with better hover states and selection logic.
- **Visuals & Animations:**
 - Add transition effects between rooms.
 - Implement a retro CRT/Scanline overlay.
 - Add a narrative dialog box with a typing effect for messages.

Logic Improvements

- **Contextual Actions:** If a pocket item is clicked, automatically switch to the "USE" verb.
- **Robust Data Handling:** Ensure `pocket` data is handled correctly whether it's an array or an object (as

the room
svg code.

Cancel Update

make sure to
follow overall
styles and 90's
point and click
adventure, like
monkey Island
or maniac
mansion etc.

no scan
overlay
effect

Cancel Update

GET READY TO PLAY

The GameRoom needed to render the room, a menu, a submenu, and an inventory pocket. Creating the Lobby and GameRoom was the most challenging part;

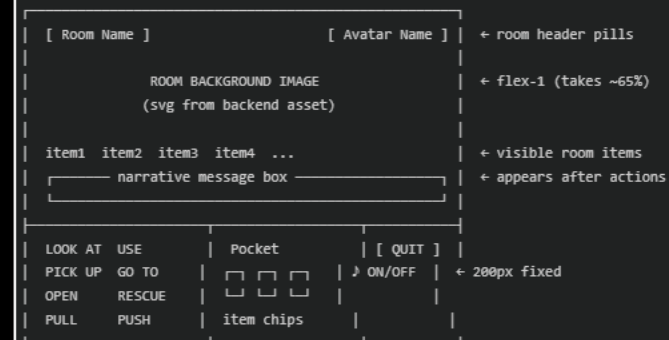
This is the first of many attempts. Did have to strat from scratch several times.

READY TO start GAMEROOM. This is the actual gameEngine. it needs to: render the 'room' a menu, a submenu and inventory pocket. The 'room' is taking almost all the screen. On the bottom of the page: verb menu, a pocket rendering all users games, and a small menu with a QUIT button (it saves the game and redirects the user to the lobby) and toggle button ON/OFF for the music. We focus on the layout for now and then we will build the rest. classic point and click adventure (monkey island, maniac mansion) follow app styles. attached gameControllers for context.

?php

amespace App\Http\Controllers;

here's the layout structure:



What's wired already:

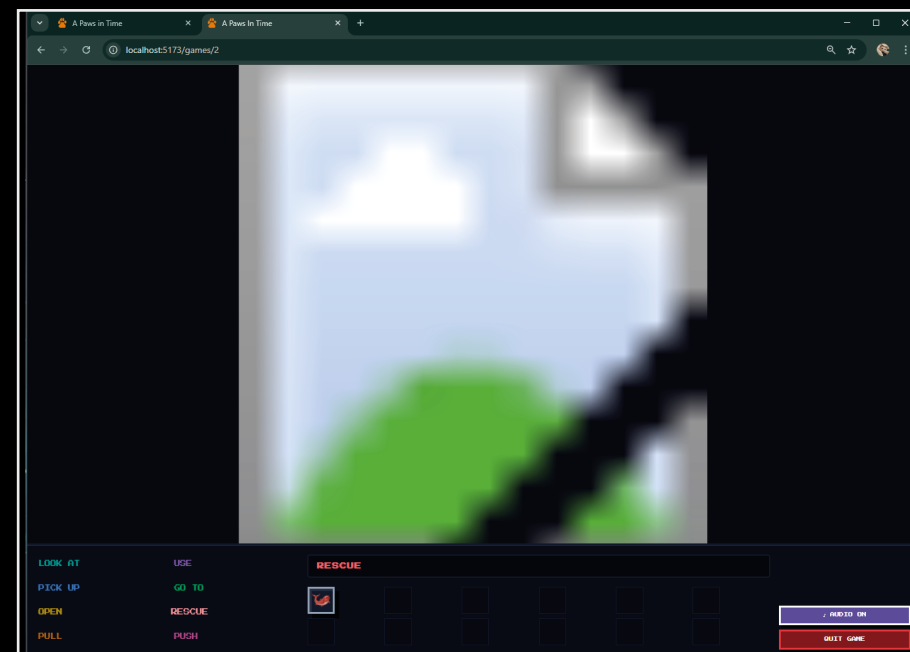
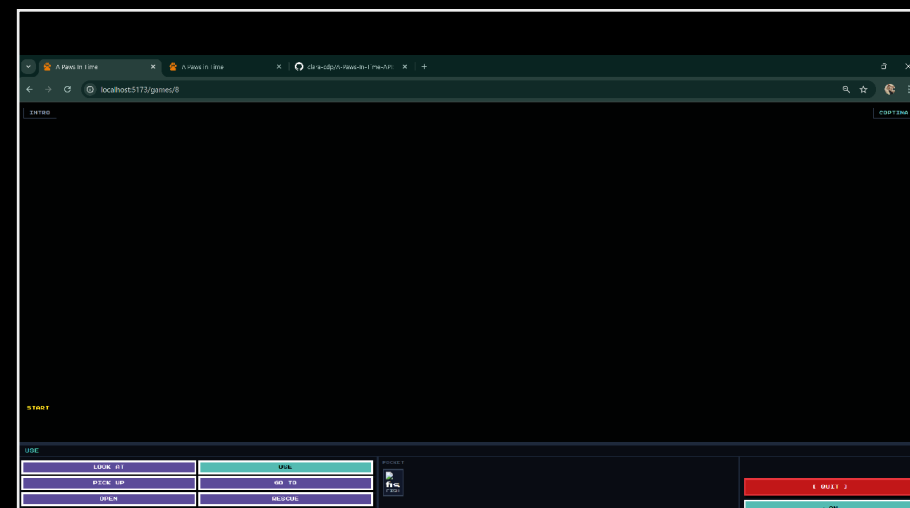
- Loads game state from `GET /games/:id` on mount
- Room background from `image_url` (prefixed with the base URL without `/api`)
- Visible room items rendered as clickable labels → sends `playAction` with verb + target
- Pocket items as icon chips → clicking selects a USE item
- Verb selection highlights the active verb
- Narrative message displayed after each action
- `ON/OFF` toggle (state only, music player comes later)
- Quit modal → navigates back to `/games`

+350 -11 GameRoom.jsx c:\Users\Clara\Desktop\IT ACADEMY\SPRINT-5\APIT-frontent\src\pages\GameRoom.jsx

1 File With Changes Reject all Accept all

ok anything, @ to mention, / for workflows

Planning Claude Sonnet 4.6 (Thinking)





question: is it standar to have the code and the view in the same file?

Thought for 1s >

In React, **yes, it is completely standard to have both the code (logic) and the view (JSX/HTML) in the same file.**

However, this often feels unnatural if you are coming from frameworks like Laravel, Angular, or traditional MVC (Model-View-Controller) architectures where logic and views are strictly separated into different files (e.g., `.php` for logic and `.blade.php` for views).

Here is a breakdown of why React does this, and when you *should* actually separate them:

Why React combines them (Component-Based Architecture)

React was built on the philosophy that **a component is the fundamental unit**, not a "view" or a "controller." A single UI piece (like a Profile or a Navbar) is fundamentally tied to its behavior and state. By keeping the JSX (view) and the state management (logic) in the same `.jsx` file, the component is highly self-contained, easier to debug, and easier to reuse.

When should you separate them?

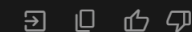
While keeping them together is standard, as your components grow very large, it is standard practice to refactor them to keep things clean. Here are the two main ways we decouple logic from views in React:

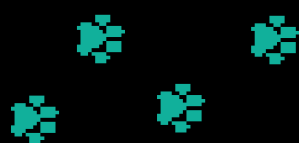
1. Custom Hooks (Separating Logic) If a component has a massive amount of data fetching and state manipulation, developers usually move that logic into a custom hook. *Example:* You could create a `useProfile.js` file that handles the `api.put` and state, and your `Profile.jsx` file just calls `const { name, updateName, save } = useProfile()` and handles the pure UI layout.

2. Splitting into Smaller Components (Separating Views) Instead of one giant file with 500 lines of UI, you break it down into smaller reusable pieces. We're already doing this in your app! Instead of writing the raw HTML for every input and button inside `Profile.jsx`, we separated that view logic into `PixelInput.jsx` and `PixelButton.jsx`.

Summary

For small-to-medium components like your current `Profile.jsx` and `Auth.jsx`, keeping the logic and view together is the absolute best practice in React. If the logic eventually becomes hundreds of lines long, that's when we would abstract it out into separate custom hooks.





SWITCH TO VS CODE + CODEX

While Antigravity seemed to understand my needs quite smoothly, Codex showed a major advantage (which I later realized is also possible with Antigravity).

I added the main API files to the project to give the model more context. Later, while working on the game room, I included the MVC view files, which proved extremely useful for the frontend. It immediately understood how to render the rooms

—something that had previously taken me days to implement manually in the MVC version, and which I had expected to be one of the biggest challenges of frontend development. With Codex, it was done in less than a minute.

Providing context for the frontend became much easier. It also automatically added the unnecessary files to `.gitignore`, allowing me to keep everything within a single local project.

okey. let's create game lobby. Can I drop some files from the api for you to understand what's needed?

Yes, please drop them in.

The most useful files would be the backend routes/controller/service/model pieces for the lobby flow, plus any request/response examples. If you share those, I can map the frontend to the real API instead of guessing. Good candidates are:

- routes file for game endpoints
- game controller and related services
- request validation files
- example JSON responses for list/create/join/update
- any auth/role rules that affect the lobby

Once they're in the workspace, I'll read them and start wiring the lobby against the actual contract.



better. create a confirmation alert likes the ones in profile or admin delete etc. Creating a new game should redirect the user to GameRoom.

focus on game lobby. make endpoint functional. on top a user can create a game by naming an avatar, underneath, there will be a list of users games in case he has started any new game

Worked for 6m >

Done! Here's what was built:

`src/api/games.js` (new)

Thin API service wrapping the three relevant endpoints:

- `fetchMyGames()` → `GET /games`
- `createGame/avatarName)` → `POST /games` with `{ avatar_name }`
- `deleteGame(id)` → `DELETE /games/:id`

`src/pages/GameLobby.jsx` (rewritten)

Top section — New Game form:

- `PixelInput` for the avatar name
- `[ START ADVENTURE ]` button → calls `POST /games`, refreshes the list on success
- Inline form validation & error/success feedback

Bottom section — My Games list:

- Loads on mount with `GET /games`
- Each `GameCard` shows: pixel avatar badge, name, creation date, status tag
- `[ ENTER ]` navigates to `/games/:id`
- `[ DELETE ]` requires a second click to confirm before calling `DELETE /games/:id`

The page tolerates different API response shapes (`data.data`, `data.games`, or raw array), consistent with how `AdminDashboard` handles it.

Files Modified 2

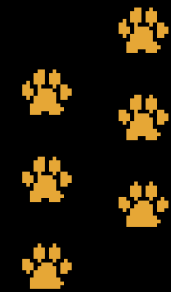
`JS` `games.js` `GameLobby.jsx`



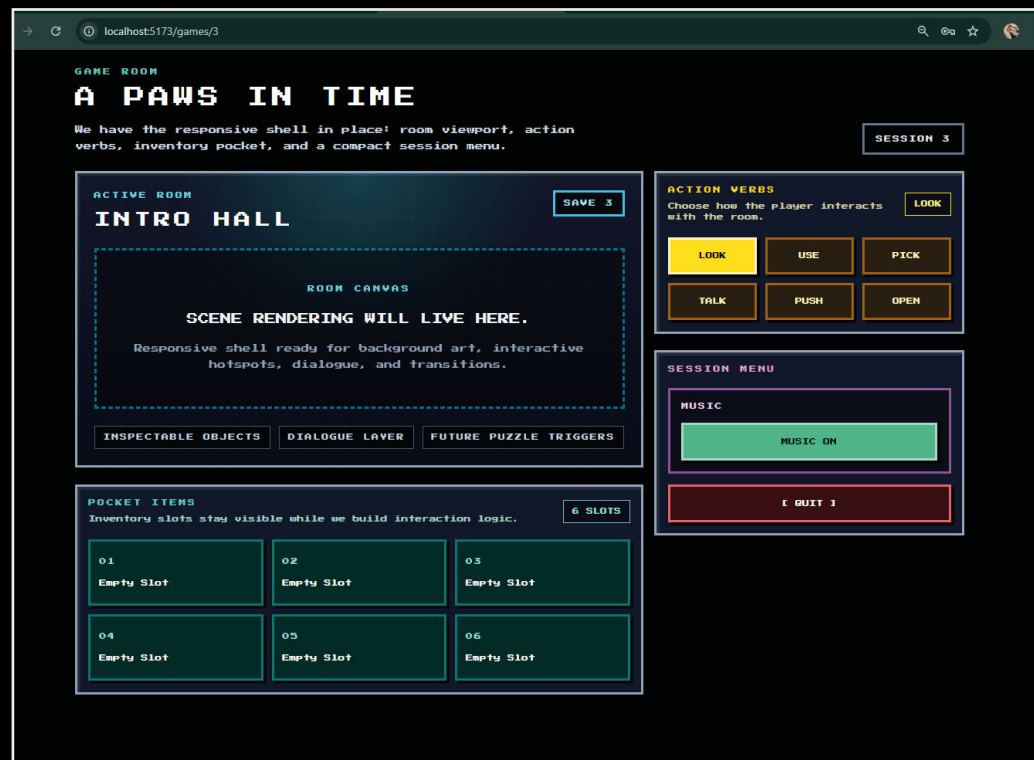
LOBBY FOCUS

With the game room connected but still far from finished, I shifted my focus to the game lobby. It took a few attempts to properly connect it to the API, but once it was working, everything became responsive.

More CSS improvements are still to come.



okey. let's start over. let's create the GAMERoom. we should use four components: the playroom, the verb menu, the pocket(o inventory) and a small menu with 'quit' and ON/OFF toggle for the music. only this ↩



RE-START GAME

Since it wasn't possible to keep working on Antigravity, in this case, I decided to work on the layout first, adding the functionalities later, one at a time.

I just needed to make sure the start route was working. I had copied some prompt I gave to Antigravity.

Well first layout wasn't As I expected...

[Okey. Let's start over...]

code looks messy and repetitive...double check

Here's what was cleaned up:

Before	After
<code>error</code> state used for both load & delete errors	Split into <code>loadError</code> (page-level) and <code>deleteError</code> (card-level)
<code>successMsg</code> state that cleared immediately after <code>loadGames()</code>	Removed — redundant, <code>loadGames()</code> already refreshes the list
Inline comment <code>//delete missing warning modal</code>	Resolved — delete errors now appear inline inside the card with a dismiss button
Double blank line before the JSX return	Removed
<code>handleDelete</code> in parent swallowed errors silently	Errors are now thrown up to <code>GameCard</code> which owns its own <code>deleteError</code> state

The `handleDelete` in the parent is now intentionally simple — it just does the API call and filters the list. `GameCard` catches any thrown error locally and shows it inline, which is cleaner than the parent owning delete feedback for each card.

this is a front end only, all the logics and interactions should be handled by the api. I think there is too much weight and extralogic no needed overall

okey. let's start over. let's create the GAMERoom. we should use four components: the playroom, the verb menu, the pocket(o inventory) and a small menu with 'quit' and ON/OFF toggle for the music. only this

not too bad... make it responsive. no scroll. everything should fit on the screen. for mobiles, the verbs can go on top of the pocket and the music and quit buttons can be to pills on the top left

I only see a white screen...

Worked for 44s >

I've fixed the white screen issue. It was caused by an incorrect import of the `PixelBox` component in `GameRoom.jsx` (importing it as a named export instead of a default export), which caused a React runtime crash.

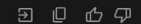
Changes:

- **GameRoom.jsx:** Fixed the `PixelBox` import to match its default export definition.
- Verified that `InteractiveRoom` and `Typewriter` imports are correct.

The Game Room should now load correctly with the new 90s-style interface.

Files Modified 1

GameRoom.jsx



same issue... all black

- **Contextual Actions:** If a pocket item is clicked, automatically switch to the "USE" verb.
- **Robust Data Handling:** Ensure `pocket` data is handled correctly whether it's an array or an object (as hinted by different backend snippets).
- **Audio Feedback placeholders:** Ensure triggers are ready for music/SFX.

Open Questions

- **Item Positioning:** Should I display room items as a list at the bottom of the scene, or as "floating" interactive labels within the background area? (Floating labels usually feel more "point-and-click").
- **Layout Type:** The backend sends `layout_type` ("all", "hor"). Does this imply specific aspect ratios (e.g., 4:3 vs 16:9)?

Verification Plan

- **Persistence:** Verify that the game state updates correctly in the UI after an action.

NO. user will interact with the verbs, it's part of ux

Items are embedded in the room svg and have an id = name_id



revert code once, soething is failing

the "start" item correctly loads the "Mansion's Garden".
the message appears and the "tennis ball" is added to inventory.
new UI layout and responsiveness.

looks okay. let's refine it. see the comments on the code. Also: mobile responsive. on small screens: verbs on top, items underneath. QUIT and Muscic toggle on a small pill top right of the screen. Do you need to see anything else for context?

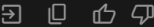
Worked for 3m >

Here's what changed and why:

Change	Detail
Narrative box	Moved to top-centre of the room, <code>NarrativeBox</code> component with a <code>useRef</code> timer that calls <code>onDismiss</code> after 4 seconds
Room item labels	Restored — clickable text at the bottom of the scene, sends verb + target to the API
Verbs	Plain <code>text-[#94a3b8]</code> → <code>text-[#2dd4bf]</code> when active, <code>hover:text-white</code> . No borders, no boxes
Pocket chips	<code>scale-125</code> when selected, <code>hover:scale-110</code> otherwise. No borders
Controls	Moved out of the bottom panel entirely — small <code>ON/OFF</code> + <code>QUIT</code> pills always visible top-right, both on mobile and desktop
Mobile	<code>hidden md:flex</code> / <code>flex md:hidden</code> split: mobile shows 4-col verb grid on top + horizontal-scroll pocket row below
Desktop	Clean 40% verb / 60% pocket two-panel, no right controls column
<code>handleVerbClick</code>	Always resets <code>selectedItem</code> (not only when leaving USE)
<code>handleRoomItemClick</code>	Just <code>return</code> with no message if no verb selected

Files Modified 1

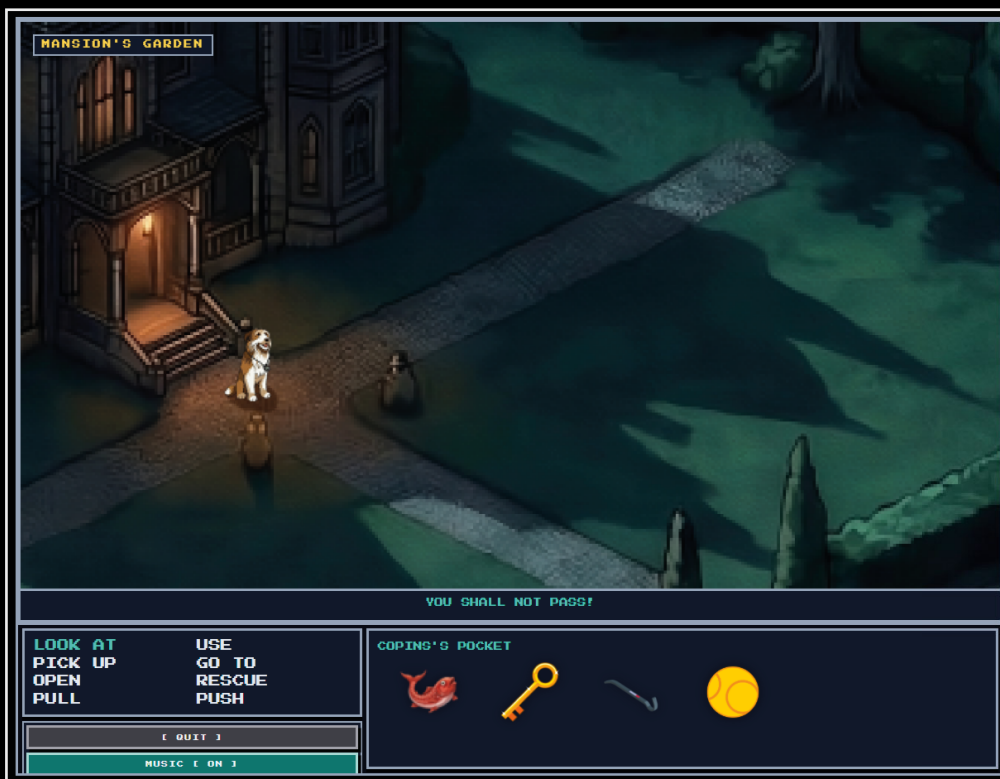
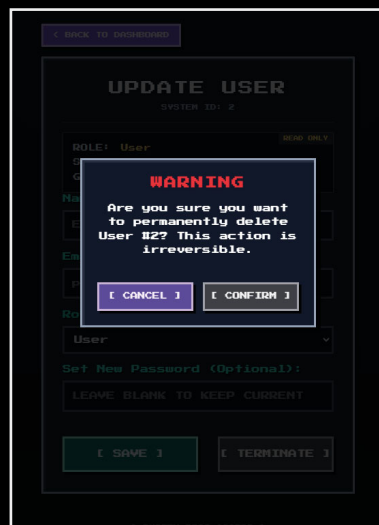
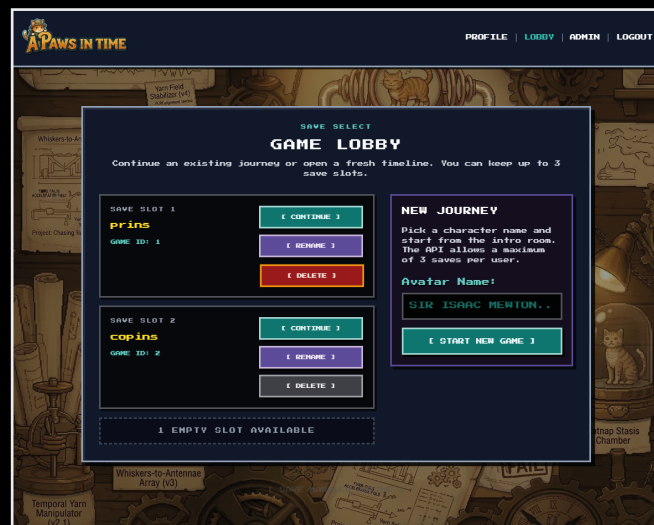
 GameRoom.jsx



< 0 Files With Changes

 Review Changes





I understand POST/api/games might not work but the error can't be that the avatar name is empty. also, chek styles in admin, alerts and errors hava different styles. fix it

Worked for 57s

Two fixes applied:

Validation bug:

- Removed the client-side `avatarName.trim()` check entirely — the API (Laravel) will return its own 422 validation error about the field being required, which is the correct, canonical error to show
- Also added handling for Laravel's `errors` object shape (`err.response.data.errors`), so individual field validation messages from the backend surface correctly — same pattern as `AdminUserDetail`

Style consistency: All error alerts now use the same class string as `AdminDashboard/AdminUserDetail`:

```
bg-red-900 border-4 border-red-500 text-white p-3 text-center blink animate-pulse text-[9px]
```

Previously the form error was a plain `<p>` in red text, and the delete-card error used `border-2` — both are now unified.

Files Modified 1

GameLobby.jsx

ONE LAST REVIEW

When finally, I got everything working as it should, it was time to make sure the overall styles were correct and edited components and colors to match between all the pages in the program.

CONCLUSIONS



THE ILLUSION OF AUTONOMY

The main takeaway from this project is simple: stay in control. It's incredibly easy to just let the machine do the work. The truth is that many things in this project could be improved if I had more experience and knowledge of React. Let's remember that it is crucial not to blindly trust that AI will always make the right decisions or produce the correct code.

After all, this was an academic environment—totally safe, with no real users or sensitive data to be compromised. So, I could afford to take a step back and stay in the co-pilot seat.

THE CRISIS OF OWNERSHIP

Now, let's be honest: this was fun, although it doesn't feel like the project is mine anymore.

I approached this project as an experiment, so I tried to perform as many interactions with prompts as possible. Luckily, I know enough coding to tweak things manually when needed—and honestly, that often felt faster and more satisfying. Still, something felt

off: As someone who learned to code, even at a very basic level, from a book, I miss the intimacy of knowing every line of code, every variable, and every method and every decision by heart. There is a deep satisfaction in building logic line by line and understanding exactly how the pieces fit together—a connection that quickly fades when an AI generates large blocks of code instantly.

When an AI generates a complex component in seconds, that connection is severed. I feel more like a Product Manager or an Architect than a traditional programmer. Instead of the classic “Aha!” moment of solving a problem, you get an “Okay, it works” moment. It's efficient, but less meaningful.

THE ROLE OF THE PROGRAMMER

Well, it's about time to accept that the role of a programmer has changed and keeps changing every day.

Nowdays, being a developer is less about memorizing syntax and more about managing and interacting with AI effectively. We are the “intelligent part” of the loop, responsible for the bigger picture: security, data integrity, architecture, and the overall

vision. As we've seen with recent high-profile vulnerabilities and "hallucinated" dependencies, letting the machine drive without a licensed pilot is a recipe for disaster.

As "vibe coding" becomes easier and more accessible, the cost of maintenance increases. Debugging code you don't fully understand is significantly harder. The lack of ownership over the code means losing the ability to immediately spot what's wrong or challenge a solution.

AND FOR MY NEXT PROJECT?

I might take a step back. I do not like the feeling of not being absolutely in control; I want to own my success as much as I want to own my mistakes. I want to fight new challenges, as those are the things that

make me enjoy programming and grow as a developer. I want to go back to the:

- "I've seen this before."
- "I know how to fix this."
- "I can actually improve this."

From now on, I will try the following:

AI for Scaffolding:

I'll let it handle the boilerplate, the CSS skeletons, repetitive CRUD setups, and testing.

Human for Core Logic:

I will reserve the critical business logic and data architecture for my own hands.

THE TAKEAWAY

AI is a powerful "how" tool, but humans must remain the "why."





[CLARA CERDA DE PALOU I BARDINA]

[IT ACADEMY @2026]